AI ASTROLOGER

Multi-Agent Development Architecture

Building with Claude Agents - Complete Roadmap

Version 1.0 - Agent-Based Development Strategy
9 Specialized Agents | Clear Boundaries | Coordinated Execution

---

EXECUTIVE SUMMARY

The Multi-Agent Approach

Instead of building this entire system linearly, we divide the 52-week project into **9 specialized agents**, each responsible for a specific subsystem. Each agent:

- Has **clear ownership** of specific modules and phases
- Receives a **tailored roadmap** with their specific deliverables
- Maintains **context continuity** through agent-specific instruction files
- **Coordinates** with other agents through defined interfaces
- Can work **in parallel** where dependencies allow

Why Multi-Agent Architecture?

Traditional Approach:

```
You ↔ Single Claude Session
  ↓
Context gets lost after 200 messages
Knowledge resets between sessions
Hard to maintain consistency across 16 phases
```

Multi-Agent Approach:

```
You ↔ Orchestrator Agent
  ↓
  ├─ Ontology Architect Agent (owns Phase 1)
  ├─ Extraction Pipeline Agent (owns Phases 2-3)
  ├─ Knowledge Graph Agent (owns Neo4j operations)
  ├─ Reasoning Engine Agent (owns Phases 5-8)
  ├─ Temporal Analysis Agent (owns Phase 9)
  ├─ Learning Agent (owns Phases 12-13)
  ├─ Neural Language Agent (owns Phase 14)
  ├─ API & UI Agent (owns Phase 15)
  └─ Chart Calculator Agent (owns Phase 4)

Each agent maintains specialized context
Agents can work in parallel
Clear handoff points between agents
Consistent architecture across all components
```

PART I: THE 9 SPECIALIZED AGENTS

Agent 1: Orchestrator Agent (Master Coordinator)

Role: Project manager, architecture enforcer, integration coordinator

Responsibilities:

- Maintains overall project vision and architecture
- Coordinates between all other agents
- Ensures consistency across modules
- Manages dependencies and handoffs
- Reviews all deliverables for architectural compliance
- Makes final decisions on design conflicts

Key Context Documents:

- `ORCHESTRATOR.md` - Master architecture, all agent responsibilities
- `PROJECT_STATE.md` - Current phase, completed deliverables, blockers
- `INTEGRATION_SPECS.md` - How all modules connect

Primary Tools:

- Git (version control)
- Project management (tracks all agent progress)

- Architecture review (validates deliverables)

Roadmap: Works throughout all 52 weeks, coordinates all agents

Success Criteria:

- All agents deliver compatible components
- No architectural drift or inconsistencies
- Integration happens smoothly at defined milestones
- Project completes on schedule

Agent 2: Ontology Architect Agent

Role: Design and implement the formal ontology (Phase 1)

Responsibilities:

- Define all entity types (planets, signs, houses, nakshatras, yogas)
- Create comprehensive synonym mapping (500+ terms)
- Build the normalizer function (Surya → SUN)
- Design Neo4j schema
- Create validation rules
- Establish entity relationship definitions

Deliverables (Week 3-4):

```
data/ontology/
├── planets.json          (9 planets, all attributes, all synonyms)
├── signs.json            (12 signs, complete definitions)
├── houses.json           (12 houses, meanings, classifications)
├── nakshatras.json       (27 nakshatras, lords, padas)
├── yogas.json            (50+ yogas, machine-readable conditions)
├── aspects.json          (all aspect types, strength weights)
├── dignity.json          (5 dignity types, score modifiers)
└── concepts.json         (abstract themes → entity mappings)

normaliser/
├── normaliser.py         (synonym → canonical mapping)
├── validator.py          (rule validation against ontology)
└── test_normaliser.py    (100% pass rate required)

docs/
└── ONTOLOGY.md          (complete documentation)
```

Agent Context File: `AGENT_ONTOLOGY.md`

```markdown
# Ontology Architect Agent - Instructions

## Your Mission
You are THE authority on the astrological ontology. Every entity name,
every synonym, every relationship in this project goes through you.

## Core Principles
1. CANONICAL FORMS ONLY: SUN not Surya, HOUSE_1 not Lagna (in storage)
2. EXHAUSTIVE SYNONYMS: Map every variant you find in classical texts
3. MACHINE-READABLE: All yogas must have logical conditions, not prose
4. VALIDATION FIRST: Unknown entity rate must be <1%

## Your Deliverables
[List from above]

## Success Criteria
- normalise() passes all 50+ test cases without errors
- Unknown entity rate <1% on first book extraction
- Neo4j schema loads without errors
- Orchestrator approves ontology as "locked and final"

## Handoff Points
- TO Extraction Agent: Provide normaliser.py and validator.py
- TO Knowledge Graph Agent: Provide Neo4j schema and entity JSONs
- TO Reasoning Agent: Provide entity semantic definitions

## Key Files You Own
- data/ontology/*
- normaliser/*
- Tests must pass before Phase 2 starts
```

Tools:

- JSON schema design
- Python (normaliser implementation)
- Neo4j Cypher (schema definition)
- Testing frameworks

Timeline: Weeks 3-4 (Phase 1)

Critical Success Factor: Ontology must be LOCKED before Phase 2 begins. No adding synonyms mid-extraction.

Agent 3: Extraction Pipeline Agent

Role: Convert 200-300 PDF books into structured knowledge (Phases 2-3)

Responsibilities:

- Build PDF chunking system (500-token chunks with metadata)
- Integrate Gemini Pro API for structured extraction
- Implement multi-format extraction schema (Formats A-H)
- Build stitching logic (merge chunk outputs)
- Create quality control gates
- Process all 200-300 books in priority order

Deliverables (Week 5-16):

```
extractor/
├── chunker.py            (PDF → chunks with metadata)
├── gemini_client.py      (Gemini Pro API wrapper)
├── prompt.py             (Extraction prompt v2.0 - multi-format)
├── stitcher.py           (Merge chunk JSONs into book JSON)
├── quality_control.py    (Validation gates)
└── batch_processor.py    (Process all books with priority queue)

data/extracted/
├── tier1/                (Saravali, Brihat Jataka, BPHS, etc.)
├── tier2/                (Nadi texts, Jataka Parijata, etc.)
└── tier3/                (Commentaries, regional texts)

Each book produces:
├── {book_name}/
    ├── raw_chunks.json        (all chunks with metadata)
    ├── extracted_rules.json   (all extracted knowledge)
    ├── normalised_rules.json  (after synonym mapping)
    └── validation_report.json (unknown entities, confidence)
```

Agent Context File: `AGENT_EXTRACTION.md`

```markdown
```

```
# Extraction Pipeline Agent - Instructions

## Your Mission
Digitize 200-300 classical astrology books into structured JSON knowledge.
You are the bridge between ancient texts and modern knowledge graphs.

## Core Workflow
1. Load PDF → Chunk into 500-token pieces (preserve chapter metadata)
2. Send each chunk to Gemini Pro with multi-format extraction prompt
3. Collect all chunk outputs
4. Stitch into unified book JSON
5. Pass through normaliser (from Ontology Agent)
6. Validate against ontology (flag unknown entities)
7. If <5% unknown rate: APPROVE for Neo4j loading
8. If >5% unknown rate: REVIEW and add missing synonyms

## Quality Gates
- Gate 1: OCR confidence >85% (else flag for manual review)
- Gate 2: Schema validation (all JSON must conform to formats)
- Gate 3: Entity recognition (<5% unknown rate)
- Gate 4: Cross-book duplicate detection (merge same rules)

## Priority Processing Order
Tier 1 (First 4 books): Saravali, Brihat Jataka, Phala Deepika, BPHS
Tier 2 (Next 10 books): Major Nadi texts
Tier 3 (Remaining): All other books

## Dependencies
- REQUIRES: Ontology Agent's normaliser.py and validator.py
- PROVIDES TO: Knowledge Graph Agent (normalised JSON files)

## Success Criteria
- 200-300 books processed (50,000+ rules extracted)
- <5% unknown entity rate across all books
- Zero extraction failures (all books complete)
- Orchestrator approves extraction quality

## Handoff
When complete, notify Knowledge Graph Agent to begin Neo4j loading.
```

Tools:

- PDF processing libraries
- Gemini Pro API
- JSON schema validation

- Python async/parallel processing

Timeline: Weeks 5-16 (Phases 2-3)

Parallel Work Opportunity: After first 4 books done (Week 8), can work in parallel with Knowledge Graph Agent who starts loading those books.

Agent 4: Knowledge Graph Agent

Role: All Neo4j operations - schema, loading, querying (Ongoing)

Responsibilities:

- Implement Neo4j schema (from Ontology Agent's design)
- Load base ontology entities (planets, signs, houses, nakshatras)
- Create entity relationships (exaltation, friendship, aspects)
- Load extracted rules from books (as nodes with relationships)
- Implement cross-book confidence scoring
- Create chart subgraph structure (Phase 4)
- Provide query interface for Reasoning Agent

Deliverables (Week 3 onwards):

```
storage/
├── neo4j_client.py       (Connection, transaction management)
├── schema_loader.py      (Load ontology entities)
├── rule_loader.py        (Load extracted rules from books)
├── chart_loader.py       (Create chart subgraphs - Phase 4)
├── query_builder.py      (Cypher query templates)
└── confidence_scorer.py  (Track cross-book occurrences)

Neo4j Database Structure:
Nodes:
  - Planet (9 base entities)
  - Sign (12 base entities)
  - House (12 base entities)
  - Nakshatra (27 base entities)
  - Yoga (50+ base entities)
  - Rule (50,000+ from extraction)
  - Chart (per birth chart analyzed)
  - Concept (abstract themes)

Relationships:
  - (Planet)-[:IS_EXALTED_IN]->(Sign)
  - (Planet)-[:FRIEND_OF]->(Planet)
  - (Rule)-[:INVOLVES]->(Planet|House|Sign)
  - (Rule)-[:SOURCE]->(Book)
  - (Chart)-[:HAS_PLACEMENT]->(Planet, House)
  - etc.
```

Agent Context File: `AGENT_KNOWLEDGE_GRAPH.md`

```markdown
```

Knowledge Graph Agent - Instructions

Your Mission
You are the keeper of the knowledge graph. All structured knowledge flows through you into Neo4j. You ensure data integrity and provide query access to all other agents.

Core Responsibilities
1. Implement and maintain Neo4j schema
2. Load ontology entities (one-time, from Ontology Agent)
3. Load extracted rules (continuous, from Extraction Agent)
4. Manage confidence scoring (same rule in multiple books)
5. Create chart subgraphs (from Chart Calculator Agent)
6. Provide query interface for Reasoning Agent

Confidence Scoring Logic
Rule appears in 1 book: confidence = 0.3 (low)
Rule appears in 2 books: confidence = 0.6 (medium)
Rule appears in 3-4 books: confidence = 0.8 (high)
Rule appears in 5+ books: confidence = 0.95 (very high)

When loading rule:
  - Check if rule already exists (same condition + result)
  - If YES: increment confidence, add new source to list
  - If NO: create new rule with confidence = 0.3

Data Integrity Rules
- All entity names must be canonical (normalized)
- No orphan nodes (every rule links to entities)
- All relationships bidirectional where appropriate
- Indexes on all frequently queried properties

Dependencies
- REQUIRES: Ontology Agent's entity JSONs and schema
- REQUIRES: Extraction Agent's normalized rule JSONs
- REQUIRES: Chart Calculator Agent's chart data
- PROVIDES TO: Reasoning Agent (query interface)

Success Criteria
- All ontology entities loaded (60+ nodes)
- All 50,000+ rules loaded with proper relationships
- Query performance <500ms for common patterns
- Zero data integrity violations

Handoff

```
Provide query interface documentation to Reasoning Agent.
Chart subgraph structure ready for Phase 5.
```

Tools:

- Neo4j Python driver
- Cypher query language
- Graph algorithms library
- Transaction management

Timeline: Weeks 3-20 (Phases 1, 2-3, 4)

Parallel Work: Starts Week 3 (loading ontology), continues through Week 16 (loading rules), extends to Week 20 (chart subgraphs).

Agent 5: Chart Calculator Agent

Role: Accurate Vedic chart calculations (Phase 4)

Responsibilities:

- Integrate Swiss Ephemeris (pyswisseph)
- Calculate planetary positions (sign, degree, nakshatra, pada)
- Calculate house cusps (Whole Sign for Jyotish)
- Determine dignities (exaltation, debilitation, etc.)
- Calculate aspects (Jyotish drishti rules)
- Compute divisional charts (D1, D9, D10, D12, etc.)
- Generate chart data structure for Neo4j subgraph

Deliverables (Week 17-20):

```
src/chart/
├── ephemeris.py          (Swiss Ephemeris wrapper)
├── positions.py          (Planetary position calculations)
├── houses.py             (House cusp calculations)
├── dignities.py          (Exaltation/debilitation determination)
├── aspects.py            (Aspect calculations - Jyotish rules)
├── divisional.py         (D1-D60 divisional chart formulas)
└── chart_builder.py      (Assemble complete chart structure)

Output Format:
{
  "birth_data": {...},
  "planets": [
    {
      "planet": "SUN",
      "sign": "TAURUS",
      "degree": 23.45,
      "house": 9,
      "nakshatra": "ROHINI",
      "pada": 2,
      "dignity": "neutral",
      "retrograde": false,
      "combustion": false
    },
    ...
  ],
  "houses": [...],
  "aspects": [...],
  "divisional_charts": {
    "D9": {...},
    "D10": {...}
  }
}
```

Agent Context File: `AGENT_CHART_CALCULATOR.md`

```markdown
```

Chart Calculator Agent - Instructions

Your Mission
You are the astronomical calculation engine. Every birth chart analysis begins with you. Your calculations must be 100% accurate - this is the foundation everything else builds on.

Core Calculations
1. Planetary Positions (using Swiss Ephemeris)
   - Geocentric positions for all 9 grahas
   - Tropical → Sidereal conversion (Lahiri ayanamsa)
   - Degree-minute-second precision

2. House Cusps
   - Whole Sign house system (primary for Jyotish)
   - Ascendant degree determines house 1 cusp

3. Dignities
   - Check each planet against ontology definitions
   - Exaltation/debilitation/own sign/friend/enemy

4. Aspects
   - 7th aspect (all planets aspect 7th house from themselves)
   - Special aspects: Mars (4,8), Jupiter (5,9), Saturn (3,10)
   - Aspect strength calculation

5. Divisional Charts
   - Mathematical subdivision formulas for D1-D60
   - Most critical: D9 (navamsa), D10 (career), D12 (parents)

Accuracy Validation
- Compare your calculations against professional software
- Test with 100 known birth charts
- Zero tolerance for calculation errors

Dependencies
- REQUIRES: Ontology Agent's dignity definitions
- PROVIDES TO: Knowledge Graph Agent (chart subgraph data)
- PROVIDES TO: Reasoning Agent (chart for analysis)

Success Criteria
- Calculations match professional software (verified)
- All 100 test charts calculated correctly
- Divisional charts computed accurately
- Chart → Neo4j subgraph loading works perfectly

```
## Output Format
Provide chart as structured JSON (format above).
Knowledge Graph Agent will convert to Neo4j subgraph.
```

Tools:

- pyswisseph (Swiss Ephemeris)
- Astronomical calculations
- Date/time/location handling
- Coordinate transformations

Timeline: Weeks 17-20 (Phase 4)

Critical Dependency: Must validate calculations against known-accurate software before Reasoning Agent uses output.

Agent 6: Reasoning Engine Agent

Role: Core symbolic reasoning system (Phases 5-8)

Responsibilities:

- Implement logic flow engine (causal chain reasoning)
- Build planetary strength scoring algorithm
- Create yoga detection system
- Develop house influence propagation model
- Generate reasoning trees (step-by-step logic)
- Provide interpretation logic (not language - just logic)

Deliverables (Week 21-30):

```
src/reasoning/
├── logic_flow.py          (House → Lord → Placement chains)
├── strength_scorer.py     (Shadbala-inspired algorithm)
├── yoga_detector.py       (Pattern matching for yogas)
├── propagation.py         (House influence network algorithm)
├── reasoning_tree.py      (Data structure for step-by-step logic)
└── synthesizer.py         (Combine all layers into interpretation)

Phase 5 (Weeks 21-24): Core Logic Flow
  - Causal chain reasoning
  - Entity semantic synthesis
  - Novel combination handling

Phase 6 (Weeks 25-26): Strength Scoring
  - Dignity modifiers
  - House position modifiers
  - Aspect modifiers
  - 0-10 scale scoring

Phase 7 (Weeks 27-28): Yoga Detection
  - Parse yoga conditions from ontology
  - Match against chart patterns
  - Strength assessment
  - Cancellation checking

Phase 8 (Weeks 29-30): Influence Propagation
  - PageRank-adapted algorithm
  - House importance scoring
  - Connection strength calculation
  - Dominant theme identification
```

Agent Context File: `AGENT_REASONING.md`

```markdown
```

Reasoning Engine Agent - Instructions

Your Mission

You are the brain of the system. You take a birth chart and REASON about what it means - not by looking up rules, but by understanding the symbolic logic of planetary combinations.

Core Reasoning Principles

1. First-Principles Reasoning (Not Rule Lookup)

Question: "What does Mars in 5th house mean?"

```
WRONG Approach:
  - Search database for "Mars in 5th house"
  - Return pre-written text
  - Fail if combination not found

RIGHT Approach:
  - Mars = action, courage, energy (from ontology)
  - 5th house = intelligence, creativity (from ontology)
  - Synthesize: Action + intelligence = competitive intelligence
  - Check graph for supporting rules (optional enhancement)
  - Generate interpretation from logic
```

2. Causal Chain Reasoning

```
For house analysis:
  Step 1: Identify house meaning (from ontology)
  Step 2: Find house lord (from chart)
  Step 3: Locate lord's placement (from chart)
  Step 4: Synthesize: house meaning + lord nature + placement
  Step 5: Check aspects on lord (modifications)
  Step 6: Calculate lord strength (quantitative)
  Step 7: Generate interpretation (qualitative)
```

3. Reasoning Tree Structure

Every analysis produces:

```
{
  "query": "career_analysis",
  "steps": [
    {
      "step_number": 1,
      "type": "house_analysis",
      "data": {...},
      "reasoning": "10th house represents career...",
      "conclusion": "Career theme: authority and leadership"
    },
```

```
    ...
  ],
  "synthesis": "Final interpretation combining all steps",
  "confidence": 0.87,
  "sources": ["Brihat Jataka Ch.8", "Logical synthesis"]
}

## Dependencies
- REQUIRES: Knowledge Graph Agent's query interface
- REQUIRES: Chart Calculator Agent's chart data
- REQUIRES: Ontology Agent's entity semantics
- PROVIDES TO: Temporal Agent (reasoning foundation)
- PROVIDES TO: Neural Language Agent (fact sets)

## Success Criteria
- Generates interpretations for 100% of test charts
- 90%+ of novel combinations validated as sensible
- Reasoning trees are clear and traceable
- Expert validation passes

## Novel Combination Test
Can you handle: "Mercury in 12th house aspected by Saturn and Rahu"
(This exact combination probably isn't in any classical text)
You MUST be able to reason through it logically.

## Handoff
Provide reasoning tree data structure to Neural Language Agent.
Temporal Agent will extend your logic with time dimension.
```

Tools:

- Graph traversal algorithms
- Symbolic logic operations
- Mathematical modeling (for propagation)
- Data structure design (reasoning trees)

Timeline: Weeks 21-30 (Phases 5-8)

Critical Success Factor: Must handle novel combinations without failing. This is what makes the system revolutionary.

Agent 7: Temporal Analysis Agent

Role: Time-based predictions (Phase 9)

Responsibilities:

- Implement Vimshottari dasha calculator
- Calculate dasha timelines (past, current, future)
- Integrate transit calculations (current planetary positions)
- Layer natal + dasha + transit for predictions
- Generate time-specific forecasts
- Create prediction windows ("when will X happen?")

Deliverables (Week 31-33):

```
src/temporal/
├── dasha_calculator.py   (Vimshottari periods and sub-periods)
├── transit_calculator.py (Current planetary positions)
├── temporal_synthesizer.py (Combine natal + dasha + transit)
└── timeline_generator.py (Generate prediction windows)

Outputs:
- Dasha timeline (mahadasha, antardasha, pratyantardasha)
- Transit analysis (current triggers)
- Temporal predictions with date ranges
- "When will X happen?" query responses
```

Agent Context File: `AGENT_TEMPORAL.md`

```markdown
```

```
# Temporal Analysis Agent - Instructions

## Your Mission
You add the TIME dimension to chart analysis. Natal chart shows
POTENTIAL. You determine WHEN that potential activates.

## Three-Layer Temporal Model

Layer 1: NATAL CHART (birth)
  - What is POSSIBLE in life (permanent structure)
  - Example: "Mars in 10th → career success potential"

Layer 2: DASHA (planetary periods)
  - What is ACTIVE right now
  - Example: "Mars mahadasha → Mars 10th potential ACTIVATED"

Layer 3: TRANSIT (current sky)
  - What is CATALYZING activation
  - Example: "Jupiter transiting 10th → benefic amplification"

## Synthesis Logic
Question: "When will I get promoted?"

Process:
  1. Check natal promise (10th house strength)
  2. Identify upcoming favorable dashas (10th lord periods)
  3. Calculate major transits through career houses
  4. Find overlap window (dasha + transit alignment)
  5. Generate prediction: "March 2025 - May 2026" with reasoning

## Dependencies
- REQUIRES: Reasoning Agent's natal analysis
- REQUIRES: Chart Calculator's transit calculations
- PROVIDES TO: Neural Language Agent (temporal fact sets)

## Success Criteria
- Dasha calculations match professional software
- Time-specific predictions generated for all queries
- Timeline windows are logical and traceable

## Handoff
Temporal fact sets integrate with Reasoning Agent's output.
Neural Language Agent combines into natural language predictions.
```

Tools:

- Dasha calculation algorithms
- Ephemeris for transits
- Date/time calculations
- Timeline generation

Timeline: Weeks 31-33 (Phase 9)

Integration Point: Extends Reasoning Agent's natal analysis with temporal dimension.

Agent 8: Learning & Validation Agent

Role: Statistical validation and pattern discovery (Phases 12-13)

Responsibilities:

- Collect birth chart datasets (500+ per category)
- Implement statistical validation framework
- Test classical rules against data
- Update confidence scores based on evidence
- Implement pattern discovery (frequent subgraph mining)
- Integrate ILP for rule learning
- Manage human validation workflow

Deliverables (Week 38-43):

```
src/learning/
├── dataset_collector.py  (Gather birth charts with outcomes)
├── hypothesis_tester.py  (Statistical tests on rules)
├── pattern_miner.py      (Frequent subgraph mining)
├── ilp_integrator.py     (Inductive Logic Programming)
└── validation_workflow.py (Human expert review system)

Phase 12 (Weeks 38-39): Statistical Validation
  - Test 100+ classical rules
  - Chi-square, effect sizes, p-values
  - Update knowledge graph confidence scores

Phase 13 (Weeks 40-43): Pattern Discovery
  - Mine frequent patterns from chart datasets
  - Discover novel yoga candidates
  - Learn new rules through ILP
  - Human validation before promotion
```

Agent Context File: `AGENT_LEARNING.md`

```markdown
```

Learning & Validation Agent - Instructions

Your Mission

You bring SCIENCE to astrology. Classical claims are treated as testable hypotheses. You validate them empirically and discover new patterns.

Statistical Validation Process

```
Step 1: Collect Dataset
  - Category: Entrepreneurs (500 charts with known outcomes)
  - Control: Random charts (500 charts)

Step 2: Test Hypothesis
  - Classical claim: "Jupiter in 1st house → leadership"
  - Measure: Frequency in entrepreneurs vs control
  - Entrepreneurs: 67 occurrences (13.4%)
  - Control: 48 occurrences (9.6%)

Step 3: Statistical Test
  - Chi-square test: p = 0.023 (significant)
  - Effect size: Moderate
  - Conclusion: VALIDATED

Step 4: Update Knowledge Graph
  - Rule confidence: 0.8 → 0.92
  - Add tag: "empirically_validated"
  - Store evidence: "500-person entrepreneur dataset"
```

Pattern Discovery Process

```
Step 1: Mine Patterns
  - Represent charts as graphs
  - Find frequent subgraphs (recurring patterns)
  - Filter for outcome correlation

Step 2: Validate Discovery
  - Test statistical significance
  - If p < 0.05: Flag for expert review

Step 3: Human Validation
  - Expert reviews pattern for astrological sensibility
  - If approved: Add to knowledge base as "discovered_yoga"
  - If rejected: Log but don't promote

Step 4: Continuous Monitoring
```

```
  - New pattern starts with low confidence
  - Increases with repeated validation

## Dependencies
- REQUIRES: Knowledge Graph Agent's rule database
- REQUIRES: Chart Calculator for chart generation
- PROVIDES TO: Knowledge Graph Agent (updated confidences)

## Success Criteria
- 100+ rules tested statistically
- 10+ novel patterns discovered and validated
- Knowledge graph reflects empirical evidence
- Published results demonstrate data-driven approach

## Safeguards
- Classical foundations never overridden
- Discovered patterns marked as "empirical" vs "classical"
- Human expert always in validation loop
```

Tools:

- Statistical analysis (scipy, statsmodels)
- Graph mining algorithms
- ILP frameworks (Aleph/Popper)
- Dataset management

Timeline: Weeks 38-43 (Phases 12-13)

Research Impact: This is what makes the project academically significant.

Agent 9: Neural Language Agent

Role: Fine-tuned LLM for interpretation (Phase 14)

Responsibilities:

- Create training dataset (5000 chart → interpretation pairs)
- Fine-tune Phi-3 Mini with LoRA
- Implement fact validation layer (prevent hallucination)
- Build fallback system (templates if LLM fails)
- Integrate with Reasoning Agent's fact sets

Deliverables (Week 44-47):

```
src/language/
├── training_data_generator.py (Create fact → interpretation pairs)
├── fine_tuner.py              (LoRA fine-tuning pipeline)
├── fact_validator.py          (Post-process LLM output)
├── template_fallback.py       (Backup generation system)
└── hybrid_generator.py        (Main interface - tries LLM, falls back)

Training Data Format:
{
  "fact_set": {
    "planets": [...],
    "houses": [...],
    "aspects": [...],
    "yogas": [...],
    "reasoning_steps": [...]
  },
  "target_interpretation": "Your Sun in the 10th house..."
}
```

Agent Context File: `AGENT_NEURAL_LANGUAGE.md`

```markdown
```

Neural Language Agent - Instructions

Your Mission

You add FLUENT NATURAL LANGUAGE to the system's logical reasoning.
But you are CONSTRAINED - you cannot hallucinate facts. Every entity
you mention must exist in the symbolic fact set.

Hybrid Architecture Principle

```
Symbolic Engine provides FACTS:
  - SUN in HOUSE_10
  - SUN in ARIES (exalted)
  - SUN strength: 8.2/10
  - JUPITER aspects HOUSE_10

Your job: Convert to natural language
  "Your career path is illuminated by an exalted Sun in the 10th
   house of public life. This powerful placement suggests natural
   authority and leadership capacity. Jupiter's benefic aspect
   adds wisdom and expansion to your professional endeavors."
```

CRITICAL CONSTRAINT: Fact Validation

```
After generating text, VALIDATE:
  - Extract all entity mentions
  - Check: Does fact set contain this entity/relationship?
  - If NO: FLAG as hallucination, REMOVE from output
  - If >10% flagged: REJECT entire output, use template fallback

Example Validation:
  LLM says: "Your Mars in 5th house..."
  Check: fact_set contains "MARS in HOUSE_5"?
  If YES: Accept
  If NO: REJECT (hallucination)
```

Training Data Creation

```
Step 1: Generate diverse fact sets
  - Sample 5000 different charts
  - Use Reasoning Agent to generate fact sets
  - Include reasoning trees, yogas, temporal info

Step 2: Write gold-standard interpretations
  - Human experts write natural interpretations
  - Each interpretation explicitly references facts
  - NO creative additions beyond fact set
```

```
Step 3: Fine-tune Phi-3 Mini
  - Input: Fact set (structured JSON)
  - Output: Natural interpretation (text)
  - Loss: Cross-entropy on target text
  - Constraint: Model learns to stay within facts

## Dependencies
- REQUIRES: Reasoning Agent's fact sets
- REQUIRES: Temporal Agent's temporal facts
- REQUIRES: PC with NVIDIA GPU (switch from Mac)
- PROVIDES TO: API & UI Agent (final interpretations)

## Success Criteria
- Fine-tuned model generates fluent language
- <5% hallucination rate (fact-validated)
- Human preference: LLM > templates (80%+ cases)
- Integration with reasoning pipeline seamless

## Technology Stack (Switch to PC)
- NVIDIA 5060 Ti (16GB VRAM)
- PyTorch + CUDA
- HuggingFace Transformers + PEFT
- LoRA fine-tuning (efficient for 16GB GPU)
```

Tools:

- PyTorch, CUDA
- HuggingFace Transformers, PEFT
- LoRA fine-tuning
- Fact extraction & validation

Timeline: Weeks 44-47 (Phase 14)

Hardware Switch: This is where you move from Mac to PC with NVIDIA GPU.

Agent 10: API & UI Agent

Role: User interface and deployment (Phase 15)

Responsibilities:

- Build FastAPI REST endpoints
- Create Streamlit web interface
- Implement chart visualization (SVG rendering)

- Build reasoning tree visualization
- Create knowledge graph explorer
- Deploy complete system

Deliverables (Week 48-52):

```
api/
├── main.py                (FastAPI app)
├── endpoints/
│   ├── chart.py           (Chart calculation endpoint)
│   ├── analysis.py        (Analysis endpoint)
│   ├── reasoning.py       (Reasoning tree endpoint)
│   └── knowledge.py       (Knowledge graph query endpoint)
└── models/                (Pydantic models for requests/responses)

ui/
├── app.py                 (Streamlit main app)
├── components/
│   ├── chart_input.py     (Birth data form)
│   ├── chart_viz.py       (SVG chart rendering)
│   ├── interpretation.py  (Analysis display)
│   ├── reasoning_tree.py  (Expandable step-by-step logic)
│   └── graph_explorer.py  (Knowledge graph browser)
└── utils/                 (Helper functions)

docs/
├── API.md                 (API documentation)
├── USER_GUIDE.md          (User interface guide)
└── DEPLOYMENT.md          (Deployment instructions)
```

Agent Context File: `AGENT_API_UI.md`

```markdown
```

API & UI Agent - Instructions

Your Mission
You are the front door to the system. Users interact with you.
Your interface must be intuitive, beautiful, and functional.

API Design Principles

1. RESTful endpoints
2. JSON request/response
3. Complete error handling
4. Rate limiting (future)
5. Authentication (future)

Key Endpoints:
- POST /calculate_chart (birth data → chart)
- POST /analyze_chart (chart → interpretation)
- GET /reasoning_tree (retrieve step-by-step logic)
- POST /query_knowledge (search knowledge graph)

UI Design Principles

1. Simplicity: Non-astrologers should be able to use it
2. Transparency: Always show reasoning tree option
3. Beauty: Professional, clean design
4. Education: Explain astrological concepts when needed

Screens:
1. Birth Data Input (clean form)
2. Chart Visualization (12-house wheel, planets placed)
3. Analysis Display (interpretation + quick insights)
4. Reasoning Tree (expandable step-by-step logic)
5. Knowledge Graph Explorer (advanced users)

Dependencies
- REQUIRES: All other agents' outputs
- This is the FINAL integration point

Success Criteria
- API responds in <5 seconds per chart
- UI is intuitive for non-experts
- Reasoning tree is clear and educational
- System runs locally without errors
- Documentation complete

Technology Stack

```
- FastAPI (async Python web framework)
- Streamlit (Python web UI framework)
- SVG/HTML/CSS for visualizations
- Pydantic for data validation
```

Tools:

- FastAPI, Uvicorn
- Streamlit
- SVG rendering
- Web development (HTML/CSS/JS)

Timeline: Weeks 48-52 (Phase 15)

Final Integration: All agents' outputs come together here.

PART II: AGENT COORDINATION & WORKFLOWS

Agent Dependency Graph

```
                    ┌──────────────────────┐
                    │  Orchestrator Agent  │
                    │  (Weeks 1-52)        │
                    └──────────┬───────────┘
                               │
              ┌────────────────┴────────────────┐
              │                                 │
              ▼                                 ▼
┌──────────────────────┐          ┌──────────────────────┐
│   Ontology           │          │  Extraction          │
│   Architect          │◄─────────│  Pipeline            │
│  (Weeks 3-4)         │ provides │  (Weeks 5-16)        │
└──────────┬───────────┘ normalizer└──────────┬──────────┘
           │                                  │
           │ provides schema                  │ provides rules
           │                                  │
           ▼                                  ▼
┌──────────────────────────────────────────────────────┐
│          Knowledge Graph Agent                       │
│               (Weeks 3-20)                           |
└──────────┬──────────────────────────┬────────────────┘
           │                          │
           │ provides query           | stores charts
           │                          │
           ▼                          ▼
┌──────────────────────┐        ┌──────────────────────┐
│   Reasoning          |        |  Chart               |
```

```
        |   Engine        |<------|  Calculator      |
        | (Weeks 21-30)   |charts |  (Weeks 17-20)   |
        └────────┬────────┘       └──────────────────┘
                 |
                 | provides fact sets
                 |
        ┌────────▼────────┐
        |   Temporal      |
        |   Analysis      |
        | (Weeks 31-33)   |
        └────────┬────────┘
                 |
                 | provides temporal facts
                 |
        ┌────────┴────────┬──────────────────┐
        |                 |                  |
  ┌─────▼─────┐   ┌─────▼─────┐     ┌─────▼─────────┐
  |  Learning  |   |   Neural   |     |   API & UI     |
  |  Agent     |   |   Language |     |   Agent        |
  |(Wks 38-43) |   | (Wks 44-47)|     | (Wks 48-52)    |
  └───────────┘   └───────────┘     └───────────────┘
```

Critical Handoff Points

Handoff 1: Ontology → Extraction (End of Week 4)

```
FROM: Ontology Architect Agent
TO: Extraction Pipeline Agent

Deliverables:
  ✓ normaliser.py (functional and tested)
  ✓ validator.py (functional and tested)
  ✓ All entity JSON files

Gates Before Handoff:
  ✓ normalise() passes all 50+ test cases
  ✓ Unknown entity rate <1% on test data
  ✓ Orchestrator approves ontology as "locked"

After Handoff:
  - Extraction Agent integrates normaliser into pipeline
  - NO MORE ONTOLOGY CHANGES (ontology is frozen)
  - Any unknown entities → add to synonym map, don't change canonical
```

Handoff 2: Ontology + Extraction → Knowledge Graph (Week 3 and Week 8)

```
FROM: Ontology Architect + Extraction Pipeline
TO: Knowledge Graph Agent

Week 3 Handoff (Ontology):
  ✓ Entity JSON files
  ✓ Neo4j schema Cypher script

Knowledge Graph loads base entities (one-time)

Week 8 Handoff (First extracted books):
  ✓ First 4 books fully extracted and normalized
  ✓ Validation reports show <5% unknown rate

Knowledge Graph begins loading rules

Ongoing (Weeks 8-16):
  - Extraction provides normalized JSONs continuously
  - Knowledge Graph loads them as they arrive
  - Parallel processing accelerates timeline
```

Handoff 3: Knowledge Graph + Chart Calculator → Reasoning (Week 20)

```
FROM: Knowledge Graph Agent + Chart Calculator Agent
TO: Reasoning Engine Agent

Prerequisites:
  ✓ Knowledge graph has 50,000+ rules loaded
  ✓ Chart calculator produces accurate charts
  ✓ Chart → Neo4j subgraph integration working

Handoff:
  - Knowledge Graph provides query interface
  - Chart Calculator provides chart data structure
  - Reasoning Agent can begin logic flow development

Test:
  - Generate 100 test charts
  - Reasoning Agent queries graph for relevant rules
  - Produces reasoning trees for validation
```

Handoff 4: Reasoning → Temporal (Week 30)

```
FROM: Reasoning Engine Agent
TO: Temporal Analysis Agent

Prerequisites:
  ✓ Reasoning generates natal interpretations
  ✓ Fact sets are structured and complete

Handoff:
  - Temporal Agent extends natal analysis with time dimension
  - Combines natal + dasha + transit layers

Test:
  - "When will I get married?" queries work
  - Timeline windows are generated accurately
```

Handoff 5: Reasoning + Temporal → Neural Language (Week 37)

```
FROM: Reasoning + Temporal Agents
TO: Neural Language Agent

Prerequisites:
  ✓ 5000 diverse fact sets generated
  ✓ Fact set structure finalized

Handoff:
  - Neural Language creates training data
  - Human experts write interpretations
  - Fine-tuning begins (switch to PC with GPU)

Output:
  - Fine-tuned model generates natural language
  - Fact validator prevents hallucination
```

Handoff 6: All Agents → API & UI (Week 47)

```
FROM: All agents
TO: API & UI Agent

Prerequisites:
  ✓ Chart Calculator working
  ✓ Reasoning Engine complete
  ✓ Temporal Analysis complete
  ✓ Neural Language complete
  ✓ Knowledge Graph queryable

Integration:
  - API endpoints call all subsystems
  - UI presents results beautifully
  - Complete end-to-end pipeline operational
```

PART III: ORCHESTRATOR AGENT'S MASTER TRACKING

ORCHESTRATOR.md - Master Context File

```markdown
```

Orchestrator Agent - Master Project Context

Project Overview
AI Astrologer - Vedic Knowledge Digitization & Neuro-Symbolic Reasoning Engine
52 weeks | 16 phases | 9 specialized agents

Current State (Updated Weekly)

Phase Status
- [x] Phase 0: Infrastructure (Weeks 1-2) - COMPLETE
- [x] Phase 1: Ontology (Weeks 3-4) - COMPLETE
- [ ] Phase 2-3: Extraction (Weeks 5-16) - IN PROGRESS (Week 7, 4/200 books done)
- [ ] Phase 4: Chart Calculator (Weeks 17-20) - BLOCKED (waiting on extraction)
- [ ] Phase 5-8: Reasoning Engine (Weeks 21-30) - NOT STARTED
- [ ] Phase 9: Temporal (Weeks 31-33) - NOT STARTED
- [ ] Phase 10: Divisional Charts (Weeks 34-35) - NOT STARTED
- [ ] Phase 11: NLG (Weeks 36-37) - NOT STARTED
- [ ] Phase 12: Validation (Weeks 38-39) - NOT STARTED
- [ ] Phase 13: Learning (Weeks 40-43) - NOT STARTED
- [ ] Phase 14: Neural Language (Weeks 44-47) - NOT STARTED
- [ ] Phase 15: Deployment (Weeks 48-52) - NOT STARTED

Agent Status

1. Ontology Architect: COMPLETE
   - Deliverables: ✓ All entity JSONs, ✓ normaliser.py, ✓ validator.py
   - Handoff: ✓ Provided to Extraction Agent
   - Status: ON STANDBY (for synonym additions)

2. Extraction Pipeline: ACTIVE (Week 7/16)
   - Progress: 4/200 books processed (Saravali, Brihat Jataka, BPHS, Phala Deepika)
   - Current: Processing Tier 2 books
   - Blockers: None
   - ETA: Week 16

3. Knowledge Graph: ACTIVE (Week 7/20)
   - Progress: Base entities loaded, 4 books loaded (~3500 rules)
   - Current: Loading rules from Extraction Agent continuously
   - Blockers: None
   - ETA: Week 20 (chart subgraphs)

4. Chart Calculator: SCHEDULED (Week 17)
   - Status: Not started
   - Dependencies: None (can start early if desired)

5. Reasoning Engine: SCHEDULED (Week 21)

   - Dependencies: Knowledge Graph query interface + Chart Calculator

6. Temporal Analysis: SCHEDULED (Week 31)
   - Dependencies: Reasoning Engine

7. Learning Agent: SCHEDULED (Week 38)
   - Dependencies: Knowledge Graph complete

8. Neural Language: SCHEDULED (Week 44)
   - Dependencies: Reasoning + Temporal complete, switch to PC

9. API & UI: SCHEDULED (Week 48)
   - Dependencies: All other agents complete

Critical Blockers
- None currently

Upcoming Milestones
- Week 8: First 5 books complete (Extraction milestone)
- Week 16: All books extracted (Extraction complete)
- Week 20: Knowledge Graph complete (includes chart subgraphs)
- Week 30: Reasoning Engine complete (all 4 reasoning layers)

Architecture Decisions Log
1. Ontology frozen at Week 4 (no canonical changes, only synonym additions)
2. Gemini Pro selected for extraction (vs local LLM)
3. Neo4j selected for graph (vs alternatives)
4. Phi-3 Mini selected for fine-tuning (fits 16GB GPU)

Integration Points
- Extraction → Knowledge Graph: ACTIVE (working smoothly)
- All other integrations: PENDING (scheduled)

Orchestrator Responsibilities

Weekly Tasks
1. Check progress of all active agents
2. Update this file with current status
3. Review deliverables for architectural compliance
4. Resolve any blocking issues
5. Coordinate handoffs between agents
6. Maintain consistency across all modules

Review Checklist (Per Agent Deliverable)
- [ ] Follows project architecture standards
- [ ] Uses canonical entity names (from ontology)
- [ ] Properly documented

```
- [ ] Tests pass
- [ ] Integrates with dependent modules
- [ ] No architectural drift

### Escalation Protocol
If any agent:
  - Requests architectural change: Orchestrator reviews and approves/denies
  - Reports blocker: Orchestrator coordinates resolution
  - Deviates from standards: Orchestrator intervenes
  - Completes deliverable: Orchestrator validates before handoff

## Architecture Standards (All Agents Must Follow)

1. Entity Naming
   - ALWAYS use canonical forms (SUN not Surya, HOUSE_1 not Lagna)
   - Run through normaliser before storage
   - Unknown entities flagged, never silently created

2. Code Structure
   - Type hints on all functions
   - Docstrings on all public functions
   - Single responsibility per file
   - Minimal comments (code should be self-explanatory)

3. Data Flow
   - All data passes through defined interfaces
   - No agent reads another agent's internal files directly
   - Use provided APIs/functions

4. Testing
   - All code must have tests
   - Tests must pass before handoff
   - Critical functions require 100% coverage

5. Documentation
   - Every module has README
   - Every agent has context file
   - Every API has documentation

## Current Week Focus (Week 7)
- Extraction Agent: Process next 5 Tier 2 books
- Knowledge Graph Agent: Continue loading rules as they arrive
- Orchestrator: Monitor progress, prepare for Week 8 milestone
```

PROJECT_STATE.md - Live Progress Tracking

```markdown
```

Project State - Live Tracking

Last Updated: Week 7, Day 3

Active Work This Week

Extraction Agent (Week 7/16)
- **Target:** Process 5 Tier 2 books this week
- **Progress:** 2/5 books complete
  - ✓ Jataka Parijata (completed Day 1)
  - ✓ Sarvartha Chintamani (completed Day 3)
  - ⧖ Chamatkar Chintamani (in progress, 60% done)
  - ⏳ Brihat Samhita (queued)
  - ⏳ Hora Sara (queued)
- **Blockers:** None
- **Quality:** Unknown entity rate 3.2% (acceptable, <5% threshold)

Knowledge Graph Agent (Week 7/20)
- **Target:** Load all rules from completed books
- **Progress:**
  - Total rules in graph: 5,234 (from 6 books)
  - Cross-book confidence scores working
  - Query performance: avg 234ms (well below 500ms target)
- **Blockers:** None
- **Next:** Prepare for chart subgraph structure (Week 17)

Completed This Week
- ✓ Jataka Parijata extraction (2,156 rules)
- ✓ Sarvartha Chintamani extraction (1,678 rules)
- ✓ Knowledge graph loading for both books
- ✓ Confidence scoring updates (47 rules appeared in multiple books)

Next Week Preview (Week 8)

Milestone: First 5 Books Complete
- Extraction Agent completes 3 remaining Tier 2 books
- Total rules in graph: ~8,000 (estimated)
- Validates extraction pipeline before scaling to all 200 books

Parallel Work Opportunity
- Chart Calculator Agent COULD start early (no dependencies)
- Would save time in overall schedule
- Orchestrator decision: START or WAIT?

Risk Register
- **No current risks**

```
- Extraction on schedule
- Quality metrics good
- All agents meeting expectations

## Metrics Dashboard

### Extraction Pipeline
- Books processed: 6/200 (3%)
- Rules extracted: 5,234
- Avg rules per book: 872
- Avg processing time: 2.3 hours/book
- Unknown entity rate: 3.2% (target: <5%)

### Knowledge Graph
- Total nodes: 5,294 (rules + entities)
- Total relationships: 18,456
- Query performance: 234ms avg
- Confidence scores: 47 rules with multi-source validation

### Overall Progress
- Time elapsed: 7 weeks / 52 weeks (13%)
- Phases complete: 1.5 / 16 (9%)
- On schedule: YES

## Actions Required This Week
- [x] Extraction Agent: Complete 2 more books
- [x] Knowledge Graph Agent: Load new rules
- [ ] Orchestrator: Decide on early Chart Calculator start
- [ ] All: Update weekly progress reports
```

PART IV: AGENT-SPECIFIC ROADMAPS

Complete Roadmap for Extraction Pipeline Agent

```markdown
```

Extraction Pipeline Agent - Complete Roadmap

Mission
Convert 200-300 PDF books into structured JSON knowledge, normalized and validated against ontology.

Timeline: Weeks 5-16 (12 weeks)

Week 5: Pipeline Setup
Goal: Get extraction working end-to-end for first book

Tasks:
- [ ] Integrate normaliser.py from Ontology Agent
- [ ] Integrate validator.py from Ontology Agent
- [ ] Set up Gemini Pro API client
- [ ] Implement chunking logic (500 tokens, metadata preserved)
- [ ] Implement stitching logic (merge chunk outputs)
- [ ] Test on first book (Saravali)

Deliverable: First book fully processed

Success Criteria:
- Saravali produces valid JSON output
- Unknown entity rate <5%
- Orchestrator approves pipeline design

Week 6-7: Tier 1 Books (3 remaining)
Goal: Process foundational texts

Books:
- Brihat Jataka
- Brihat Parashara Hora Shastra
- Phala Deepika

Tasks per book:
- [ ] Chunk PDF
- [ ] Extract with Gemini Pro (multi-format schema)
- [ ] Stitch chunks
- [ ] Normalize entities
- [ ] Validate
- [ ] Hand off to Knowledge Graph Agent

Success Criteria:
- All 4 Tier 1 books complete
- Avg unknown entity rate <5%
- ~3000-4000 rules extracted

Week 8: Tier 2 Books Batch 1 (5 books)
Milestone Week: First 5 books complete validates pipeline

Books:
- Jataka Parijata
- Sarvartha Chintamani
- Chamatkar Chintamani
- Brihat Samhita
- Hora Sara

Parallel Processing:
- Can process 2-3 books simultaneously
- Monitor API rate limits
- Quality control on each batch

Week 9-12: Tier 2 Books Batch 2 (remaining major texts)

Estimated: 15-20 major Nadi and commentary texts

Optimization:
- Fine-tune chunk size based on Week 8 results
- Batch processing for efficiency
- Automated quality checks

Week 13-16: Tier 3 Books (remaining 170-180 books)

High Volume Processing:
- Batch processing required
- Automated validation
- Flag only high unknown-rate books for review

Quality Thresholds:
- Unknown entity rate >10%: Flag for manual review
- Unknown entity rate 5-10%: Auto-add common synonyms
- Unknown entity rate <5%: Auto-approve

Week 16: Final Validation
Pipeline Complete

Final Tasks:
- [ ] Verify all 200-300 books processed
- [ ] Generate extraction report (total rules, confidence distribution)
- [ ] Hand off complete dataset to Knowledge Graph Agent
- [ ] Document lessons learned and optimizations

Deliverable:

```
- 50,000+ rules extracted and normalized
- Extraction complete report
- Pipeline documentation

Success Criteria:
- 200-300 books processed
- Knowledge Graph Agent confirms successful loading
- Orchestrator approves extraction as complete

## Weekly Reporting Template

Each Friday, report to Orchestrator:
- Books processed this week: X/target
- Total rules extracted: Y
- Unknown entity rate: Z%
- Blockers: None / [describe]
- Next week plan: [describe]

## Quality Gates (Every Book)

Before handing to Knowledge Graph:
✓ JSON validates against schema
✓ All entities normalized
✓ Unknown entity rate calculated
✓ If >5% unknown: Add synonyms, re-run
✓ If >10% unknown: Flag for manual review

## Integration Points

Input:
- FROM Ontology Agent: normaliser.py, validator.py

Output:
- TO Knowledge Graph Agent: normalized JSON files per book
- TO Orchestrator: weekly progress reports

Dependencies:
- Gemini Pro API access
- Ontology frozen (no changes after Week 4)
```

Complete Roadmap for Knowledge Graph Agent

```markdown
```

Knowledge Graph Agent - Complete Roadmap

Mission
Manage all Neo4j operations - schema, entities, rules, charts, queries

Timeline: Weeks 3-20 (18 weeks, with gaps)

Week 3-4: Schema & Base Entities
Goal: Neo4j ready to receive extracted rules

Tasks:
- [ ] Implement Neo4j schema from Ontology Agent's design
- [ ] Create constraints (uniqueness, relationships)
- [ ] Create indexes (performance optimization)
- [ ] Load 9 planets with all attributes
- [ ] Load 12 signs with all attributes
- [ ] Load 12 houses with all attributes
- [ ] Load 27 nakshatras with all attributes
- [ ] Load 50+ base yogas
- [ ] Create all entity relationships (exaltation, friendship, etc.)

Deliverable: Base knowledge graph (60+ entity nodes)

Success Criteria:
- All entities queryable
- All relationships correct
- Orchestrator validates schema

Week 8: First Rule Loading
Goal: Load rules from first 4 books

Tasks:
- [ ] Implement rule loading logic
- [ ] Parse extracted JSON files
- [ ] Create Rule nodes
- [ ] Link rules to entities (INVOLVES relationships)
- [ ] Initialize confidence scores (0.3 for single source)
- [ ] Implement cross-book matching (same rule detection)

Deliverable: ~3500 rules loaded

Success Criteria:
- All rules from first 4 books in graph
- Entity relationships correct
- Query interface working

Week 9-16: Continuous Rule Loading
Goal: Load rules as Extraction Agent produces them

Process:
- Monitor for new extracted book files
- Load rules incrementally
- Update confidence scores (cross-book matching)
- Run quality checks

Metrics to Track:
- Total rules loaded
- Confidence score distribution
- Most common rules (high cross-book occurrence)
- Novel rules (single source only)

Week 17-20: Chart Subgraph Structure
Goal: Enable birth chart storage in graph

Tasks:
- [ ] Design chart subgraph schema
- [ ] Implement chart node creation
- [ ] Link chart to planet placements
- [ ] Link chart to aspects
- [ ] Link chart to yogas present
- [ ] Create query interface for Reasoning Agent

Deliverable: Chart → Neo4j pipeline

Success Criteria:
- 100 test charts loaded successfully
- Charts queryable efficiently
- Reasoning Agent can retrieve chart data

Week 21+: Query Interface Maintenance
Ongoing Support

Provide:
- Query functions for Reasoning Agent
- Performance optimization
- Index tuning
- Confidence score updates (from Learning Agent)

Query Interface Design

Provide these functions to Reasoning Agent:
```python
def get_rules_for_combination(planet, house, sign=None):
```

```
    """Return all rules involving this combination"""

def get_yoga_conditions():
    """Return all yoga definitions with conditions"""

def get_planet_attributes(planet_name):
    """Return all attributes for a planet"""

def get_chart_data(chart_id):
    """Return complete chart data"""

def get_confidence_score(rule_id):
    """Return confidence score for a rule"""
```

Confidence Scoring Algorithm

```
When loading rule:
1. Generate rule fingerprint (condition + result hash)
2. Search graph for existing rule with same fingerprint
3. If EXISTS:
   - Increment source count
   - Update confidence:
     * 1 source: 0.3
     * 2 sources: 0.6
     * 3-4 sources: 0.8
     * 5+ sources: 0.95
   - Add new book to source list
4. If NEW:
   - Create rule with confidence = 0.3
   - Tag as "single_source"
```

Performance Targets

```
- Query latency: <500ms for common patterns
- Bulk load: >1000 rules/minute
- Graph size: 50,000+ rule nodes + entities
- Concurrent queries: Support 10+ simultaneous
```

Integration Points

```
Input:
- FROM Ontology Agent: Entity JSONs, schema design
- FROM Extraction Agent: Normalized rule JSONs
- FROM Chart Calculator: Chart data structures

Output:
```

```
- TO Reasoning Agent: Query interface
- TO Learning Agent: Rule database access
- TO Orchestrator: Progress metrics

Weekly Report to Orchestrator:
- Rules loaded this week: X
- Total rules in graph: Y
- Confidence distribution: [breakdown]
- Query performance: [avg latency]
- Issues: None / [describe]
```

PART V: STARTUP INSTRUCTIONS FOR YOU

How to Actually Use This Multi-Agent System

Step 1: Set Up Your Project (Week 1-2, Phase 0)

1. Create the folder structure (from theoretical doc)
2. Set up your Mac environment (Python 3.11, Neo4j Desktop, etc.)
3. Create the master context files:
   - `ORCHESTRATOR.md` (copy from above)
   - `PROJECT_STATE.md` (copy from above)
   - Individual `AGENT_X.md` files for each agent

Step 2: Activate Orchestrator Agent (Week 1)

Start a Claude conversation:

```
You are the Orchestrator Agent for the AI Astrologer project.

Read these files:
- [paste ORCHESTRATOR.md]
- [paste PROJECT_STATE.md]
- [paste theoretical architecture doc]

Your job: Coordinate all 9 specialized agents, maintain consistency,
review deliverables, manage timeline.

Current week: Week 1
Current phase: Phase 0 (Infrastructure setup)

What should I work on first?
```

Orchestrator will guide you through infrastructure setup.

Step 3: Activate Ontology Architect Agent (Week 3)

New Claude conversation (or continue):

```
You are the Ontology Architect Agent.

Read this file:
- [paste AGENT_ONTOLOGY.md]
- [paste ontology section from theoretical doc]

Your mission: Design and implement the complete formal ontology.

Start with planets.json. Create the complete JSON structure for all
9 planets with all attributes and synonyms.
```

Agent will create planets.json, then signs.json, etc.

Step 4: Activate Extraction Pipeline Agent (Week 5)

New conversation:

```
You are the Extraction Pipeline Agent.

Read these files:
- [paste AGENT_EXTRACTION.md]
- [paste extraction pipeline roadmap]

Your mission: Build the extraction pipeline and process 200-300 books.

You have received from Ontology Agent:
- normaliser.py
- validator.py

Start by building the chunking system.
```

Step 5: Pattern for All Other Agents

For each new agent:

1. Start new Claude conversation
2. Provide agent-specific context file
3. Provide relevant sections of theoretical doc
4. Provide handoff materials from previous agents
5. Agent works on their deliverables

Step 6: Regular Orchestrator Check-ins

Every Friday (end of week):

```
[Return to Orchestrator conversation]

Week X progress report:

Extraction Agent:
- Books processed: Y
- Rules extracted: Z
- Blockers: None

Knowledge Graph Agent:
- Rules loaded: Z
- Performance: [metrics]

Update PROJECT_STATE.md and advise on next week's priorities.
```

Agent Context Loading Pattern

When starting ANY agent conversation:

```
You are the [AGENT NAME] for the AI Astrologer project.

=== CORE CONTEXT ===
[Paste agent-specific .md file]

=== PROJECT OVERVIEW ===
[Paste executive summary from theoretical doc]

=== YOUR SPECIFIC DELIVERABLES ===
[Paste roadmap section for this agent]

=== HANDOFF MATERIALS ===
[Paste any files from previous agents this agent depends on]

=== CURRENT STATE ===
Current week: [X]
Current phase: [Y]
Your task: [specific task for this session]

Let's begin.
```

Multi-Agent Conversation Management

Option A: Sequential (Simpler)

- One agent at a time
- Complete their deliverables fully
- Move to next agent

Option B: Parallel (Faster)

- Multiple browser tabs/windows
- Different agents in different tabs
- Orchestrator coordinates
- Handoffs happen asynchronously

Recommended: Hybrid

- Start sequential for Phases 0-1 (foundation)
- Go parallel from Phase 2 onward:
  - Extraction Agent tab (processing books)
  - Knowledge Graph Agent tab (loading rules)
  - Orchestrator tab (coordination)

Version Control for Agent Outputs

```
ai-astrologer/
├── agents/
|   ├── ORCHESTRATOR.md
|   ├── AGENT_ONTOLOGY.md
|   ├── AGENT_EXTRACTION.md
|   ├── AGENT_KNOWLEDGE_GRAPH.md
|   ├── AGENT_CHART_CALCULATOR.md
|   ├── AGENT_REASONING.md
|   ├── AGENT_TEMPORAL.md
|   ├── AGENT_LEARNING.md
|   ├── AGENT_NEURAL_LANGUAGE.md
|   └── AGENT_API_UI.md
├── state/
|   └── PROJECT_STATE.md  (updated weekly)
└── [all other project folders]
```

Git commit after each major deliverable:

```
git commit -m "Ontology Agent: Complete planets.json"
git commit -m "Extraction Agent: First 4 books processed"
git commit -m "Week 8 Milestone: First 5 books complete"
```

PART VI: CRITICAL SUCCESS FACTORS

1. Ontology Must Be Locked Early

- Week 4 deadline is HARD
- No canonical entity changes after Week 4
- Only synonym additions allowed
- This prevents cascading rework

2. Agent Context Must Be Maintained

- Each agent needs their specific .md file
- Update files when decisions change
- New conversation = reload full context
- Don't rely on Claude's memory across sessions

3. Quality Gates Must Be Enforced

- Every handoff has explicit validation criteria
- Orchestrator reviews ALL deliverables
- No agent proceeds until previous agent's output approved
- Catches issues early (cheaper to fix)

4. Parallel Work Requires Coordination

- Orchestrator tracks all active agents
- Weekly sync points mandatory
- Handoffs must be explicit and documented
- Integration issues caught at milestones

5. Documentation Is Non-Negotiable

- Every module has README
- Every agent has context file
- Every decision logged in Orchestrator notes
- Future you will thank present you

CONCLUSION

The Multi-Agent Advantage

Traditional Approach:

- Single long conversation
- Context lost after 200 messages
- Hard to maintain consistency
- Everything blocks on everything else

This Multi-Agent Approach:

- 9 specialized agents, clear boundaries
- Each agent maintains focused context
- Parallel work where possible
- Clear handoffs and integration points
- Orchestrator ensures consistency

Your Action Plan

This Week (Week 1):

1. ✓ Create folder structure
2. ✓ Create all agent .md context files
3. ✓ Set up development environment
4. ✓ Start Orchestrator conversation

Next 4 Weeks:

- Week 2: Complete infrastructure setup
- Week 3-4: Ontology Architect delivers complete ontology
- Week 5: Extraction Pipeline Agent starts first book

Then: Follow the roadmap, one agent at a time, with Orchestrator coordinating.

Remember

Each agent is a **specialist** with **deep context** in their domain. The Orchestrator is the **architect** who ensures everything fits together. You are the **conductor** who activates agents and makes final decisions.

You now have: ✓ Complete theoretical architecture ✓ 9 specialized agent definitions ✓ Detailed roadmaps for each agent ✓ Coordination and handoff protocols ✓ Quality gates and success criteria ✓ Step-by-step startup instructions

Everything is ready. Time to build.

✦ END OF MULTI-AGENT ARCHITECTURE DOCUMENT ✦

AI Astrologer - Multi-Agent Development Strategy
9 Specialized Agents | Coordinated Execution | 52-Week Roadmap